

CENTRE IVOIRIEN DE ROBOTIQUE

CIR-DEV-WI-01 · Suivi 6 mois

MOIS 1 — DEVOIR 1/6

ContactBook

CRM basique — Gestion de contacts clients

Projet individuel · 5 jours · 20 points · Seuil de deblocage : 17/20 (85%)

React.js	Node.js	PostgreSQL	JWT	Sequelize	Vercel + Railway
----------	---------	------------	-----	-----------	---------------------

1 · MISE EN SITUATION

Scenario

Tu viens d'être recruté comme développeur junior dans une agence de communication digitale basée à Abidjan. Ton premier jour, ton manager t'appelle :

"On a 200 contacts clients dans 4 fichiers Excel différents. Personne ne sait quelle version est la bonne. On perd des opportunités parce qu'on ne retrouve plus les contacts. Tu as 5 jours pour nous faire quelque chose de propre."

Ta mission : construire ContactBook — une application web permettant à chaque commercial de gérer ses propres contacts clients, accessible depuis n'importe quel navigateur, avec ses données sauvegardées en base de données.

Ce que l'application doit permettre

- S'inscrire et se connecter de façon sécurisée — chaque commercial a son propre espace
- Ajouter, modifier et supprimer des contacts (nom, email, téléphone, entreprise, statut, notes)
- Rechercher et filtrer ses contacts par nom, entreprise ou statut
- Accéder à l'application depuis n'importe quel endroit — elle est déployée sur le web

2 · CE QUE TU VAS CONSTRUIRE

Structure du projet

Ton repo GitHub contiendra deux dossiers principaux :

/api — Backend Node.js	/frontend — Frontend React
<pre>src/config/database.js src/config/redis.js src/models/User.js src/models/Contact.js src/migrations/ src/routes/auth.js src/routes/contacts.js src/controllers/ src/middleware/authenticate.js src/middleware/rateLimiter.js src/app.js + src/server.js tests/ Dockerfile</pre>	<pre>src/components/ContactCard.jsx src/components/ContactForm.jsx src/components/SearchBar.jsx src/components/AuthGuard.jsx src/pages/Login.jsx src/pages/Register.jsx src/pages/Contacts.jsx src/context/AuthContext.jsx src/services/api.js src/hooks/useContacts.js .env</pre>

Les routes API a implementer

Methode	Route	Description
POST	/api/auth/register	Creer un compte — retourner uniquement un message de succes
POST	/api/auth/login	Connecter — retourner le token JWT + infos utilisateur
GET	/api/auth/me	Infos de l'utilisateur actuellement connecte
GET	/api/contacts	Liste de MES contacts (filtres optionnels : ?search=&status=)
POST	/api/contacts	Creer un contact — valide les champs obligatoires
GET	/api/contacts/:id	Detail d'un contact — 403 si ce n'est pas le mien
PUT	/api/contacts/:id	Modifier un contact — 403 si ce n'est pas le mien
DELETE	/api/contacts/:id	Supprimer un contact — 403 si ce n'est pas le mien
GET	/api/health	Health check — repond { status: 'ok' }
GET	/api-docs	Documentation Swagger interactive

3 · RUBRIQUES D'EVALUATION — 20 POINTS

Comment la notation fonctionne

Chaque critere vaut 1 point. Il est soit valide (1 pt) soit non valide (0 pt) — pas de demi-point.

Pour valider un critere : le correcteur le teste directement sur ton URL de production.

Tu dois atteindre 17/20 (85%) pour debloquer le Devoir 2 du Mois 2.

En dessous de 17/20 : tu resoumets uniquement les rubriques non validees — pas tout le projet.

Rubrique 1 — Authentification & Securite

R1	Authentification & Securite	/undefined pts
☐	POST /api/auth/register : le mot de passe est hache avec bcrypt (rounds >= 12). Verifiable via la base de donnees — le champ password ne contient jamais le mot de passe en clair.	1 pt
☐	POST /api/auth/login : retourne un objet { token, user } avec un JWT signe, expirant dans 24h. Le token est verifiable sur jwt.io avec la meme cle secrete.	1 pt
☐	Le middleware authenticate bloque toute requete sur /api/contacts sans token valide et retourne exactement { success: false, error: '...' } avec le code HTTP 401.	1 pt
☐	Les variables sensibles (JWT_SECRET, DATABASE_URL) sont dans .env et absentes du repo GitHub. Le fichier .env.example est present avec les cle vides.	1 pt

Pourquoi ces criteres ?

Un mot de passe en clair en base de donnees = faille critique. Chez n'importe quel employeur, c'est un licenciement immediat.

Un JWT sans expiration ou signe avec une cle faible = n'importe qui peut forger un token valide.

Un .env commite sur GitHub = tes cle exposees publiquement. Des bots scannent GitHub en permanence.

Rubrique 2 — Base de donnees

R2	Base de donnees PostgreSQL + Sequelize	/undefined pts
☐	La structure de la base est creee via des migrations Sequelize CLI (npx sequelize-cli db:migrate) et non via sequelize.sync(). Le dossier migrations/ contient au moins 2 fichiers horodates.	1 pt

☐	Le modele Contact contient : name (STRING, obligatoire), email (STRING, valide isEmail), phone (STRING), company (STRING), status (ENUM : lead/prospect/client/inactif), notes (TEXT), userId (INTEGER, FK vers users, onDelete CASCADE).	1 pt
☐	L'association User.hasMany(Contact, { foreignKey: 'userId' }) est operationnelle. Un utilisateur A ne peut jamais lire, modifier ou supprimer les contacts d'un utilisateur B.	1 pt
☐	La migration de la table contacts ajoute un index sur la colonne userId et un index unique sur la combinaison (email, userId) pour eviter les doublons par utilisateur.	1 pt

Pourquoi ces criteres ?

sync() en production peut silencieusement supprimer des colonnes. Les migrations versionnent chaque changement de schema — c'est la pratique professionnelle obligatoire.

L'index sur userId accelere toutes les requetes GET /contacts d'un facteur 10 a 100 des que la table depasse 10 000 lignes.

Sans l'isolation userId, un bug de code pourrait exposer les contacts d'un autre utilisateur — faille IDOR (Insecure Direct Object Reference).

Rubrique 3 — API REST

R3	API REST complete	/undefined pts
☐	GET /api/contacts retourne uniquement les contacts de l'utilisateur connecte (filtre WHERE userId = req.user.id). Le parametre de recherche ?search=texte filtre sur name ET company simultanement.	1 pt
☐	POST /api/contacts valide avec express-validator : name obligatoire (non vide), email valide si fourni. Retourne { success: false, errors: [...] } avec le code 400 si validation echoue.	1 pt
☐	PUT /api/contacts/:id et DELETE /api/contacts/:id verifient que le contact appartient a l'utilisateur connecte. Retournent 403 si l'id appartient a quelqu'un d'autre, 404 s'il n'existe pas.	1 pt
☐	La documentation Swagger est accessible sur /api-docs. Elle documente au moins : register, login, GET /contacts, POST /contacts, DELETE /contacts/:id — chacune avec ses parametres et ses reponses.	1 pt

Pourquoi ces criteres ?

Un GET qui retourne tous les contacts de tous les utilisateurs = faille de securite catastrophique et tres facile a faire par inadvertance.

Une API sans validation cote serveur = n'importe qui peut injecter n'importe quelle donnee. La validation React n'est pas suffisante — elle est contournable avec Postman.

Le 403 vs 404 : retourner 404 au lieu de 403 quand le contact existe mais appartient a quelqu'un d'autre est acceptable (evite de confirmer l'existence de l'objet).

Rubrique 4 — Interface React

R4	Interface React	/undefined pts
☐	La page /contacts affiche tous les contacts de l'utilisateur sous forme de cartes ou tableau. Un champ de recherche filtre les resultats en temps reel (onChange, pas de bouton soumettre).	1 pt
☐	Le formulaire d'ajout et de modification utilise des champs controles (valeur liee a useState). Le bouton soumettre est desactive pendant l'envoi. Un message d'erreur s'affiche si l'API retourne une erreur.	1 pt
☐	AuthContext stocke le token JWT et les infos utilisateur. Le composant AuthGuard redirige vers /login si l'utilisateur n'est pas connecte. La page /login redirige vers /contacts si l'utilisateur est deja connecte.	1 pt
☐	Le service api.js utilise axios avec un intercepteur request qui ajoute automatiquement le header Authorization: Bearer TOKEN sur chaque requete. Aucune route React ne repete ce header manuellement.	1 pt

Pourquoi ces criteres ?

Un filtre qui ne fonctionne qu'avec un bouton Soumettre est une mauvaise experience utilisateur en 2025.

Un bouton non desactive pendant l'envoi = risque de double-soumission = deux contacts identiques crees.

Sans AuthGuard : un utilisateur non connecte peut acceder directement a /contacts en tapant l'URL.

Sans intercepteur axios : chaque developpeur dans l'equipe doit penser a ajouter le header — garantie d'oubli.

Rubrique 5 — Deploiement & Documentation

R5	Deploiement & Documentation	/undefined pts
☐	L'API est deployee sur Railway. L'URL publique repond correctement sur GET /api/health (retourne { status: 'ok' }) depuis n'importe quel navigateur ou outil. Les variables d'environnement sont configurees dans Railway.	1 pt
☐	Le frontend est deploye sur Vercel. Il utilise l'URL Railway (pas localhost) comme baseUrl axios. On peut creer un compte, se connecter et ajouter un contact depuis l'URL Vercel sans erreur CORS.	1 pt

☐	Le README.md contient : un titre et une courte description, l'URL Vercel et l'URL Railway, un screenshot du dashboard contacts, et les instructions pour lancer le projet en local (3 commandes maximum).	1 pt
☐	L'historique Git contient au moins 10 commits avec des messages explicites au format feat:/fix:/docs:. Pas d'un seul commit global 'projet fini' ou de code mort commite.	1 pt

Pourquoi ces criteres ?

Une application non deployee n'existe pas — aucun recruteur ou client ne peut la voir sans te demander de la lancer chez toi.

CORS mal configure = le frontend ne peut pas appeler l'API depuis Vercel. C'est le bug numero 1 du deploiement.

Un README vide ou absent = projet professionnel inexistant. C'est la vitrine de ton travail sur GitHub.

Un seul commit 'projet fini' montre que tu n'as pas travaille progressivement. Les recruteurs regardent l'historique Git.

R.	Rubrique	Criteres	Points
R1	Authentification & Securite	4 criteres	4 pts
R2	Base de donnees PostgreSQL + Sequelize	4 criteres	4 pts
R3	API REST complete	4 criteres	4 pts
R4	Interface React	4 criteres	4 pts
R5	Deploiement & Documentation	4 criteres	4 pts
	TOTAL	20 criteres	20 pts
	SEUIL DE DEBLOCAGE MOIS 2	85%	17 / 20

4 · PLANNING SUGGERE — 5 JOURS

Ce planning est un guide, pas une obligation. L'objectif : ne jamais commencer React avant que l'API soit testée dans Postman.

Jour	Matin	Après-midi	Livrable du soir
J1	Créer le repo GitHub. Initialiser /api avec npm. Installer express, sequelize, pg, bcryptjs, jsonwebtoken, dotenv. Configurer database.js et la connexion PostgreSQL.	Ecrire les migrations User et Contact. Exécuter db:migrate. Créer les routes /auth/register et /auth/login. Tester dans Postman.	<i>Register + Login fonctionnels dans Postman. Token JWT retourne. Tables créées en base.</i>
J2	Ajouter le middleware authenticate. Ecrire les routes GET et POST /contacts avec filtres et validation express-validator.	Ecrire PUT, DELETE /contacts/:id avec vérification userId. Ajouter Swagger sur toutes les routes. Tester l'ensemble dans Postman.	<i>10 routes documentées sur /api-docs. Toutes les rubriques R1, R2, R3 validées en local.</i>
J3	Initialiser le frontend React avec Vite. Installer axios, react-router-dom. Créer AuthContext, le service api.js avec intercepteur, et AuthGuard.	Construire les pages Login et Register connectées à l'API. Tester l'inscription et la connexion depuis le navigateur.	<i>Login et Register fonctionnels. Token sauvegarde dans localStorage. Redirection après connexion.</i>
J4	Construire la page Contacts : liste des contacts, barre de recherche en temps réel. Construire le composant ContactCard.	Construire le formulaire d'ajout et de modification avec validation côté client. Connecter au DELETE de l'API.	<i>Dashboard contacts complet. Ajout, modification et suppression fonctionnels depuis React.</i>
J5	Deployer l'API sur Railway : créer le projet, ajouter le plugin PostgreSQL, configurer les variables d'environnement, exécuter les migrations en production.	Deployer le frontend sur Vercel avec VITE_API_URL pointant vers Railway. Tester la chaîne complète. Rédiger le README.md.	<i>URL Railway + URL Vercel accessibles. README complet. Projet soumis sur Moodle.</i>

Regle absolue — ne pas sauter cette étape

Avant de commencer React (Jour 3), ouvre Postman et teste CHAQUE route de l'API :

- Register avec des données valides → 201 Created
- Register avec un email déjà utilisé → 409 Conflict
- Login avec les bons identifiants → 200 + token
- Login avec mauvais mot de passe → 401
- GET /contacts sans token → 401
- GET /contacts avec token → 200 + tableau (vide au début)
- POST /contacts avec token → 201 + contact créé

— DELETE /contacts/:id avec token d'un autre user → 403 ou 404
Si un seul de ces tests echoue, corriger avant de toucher React.

5 · RESSOURCES PAR RUBRIQUE

Ces ressources sont selectionnees pour aller droit au but. Lis la documentation officielle en priorite — elle est toujours plus fiable que n'importe quel tutorial.

Rubrique 1 — Authentification

Ressource	URL	Pour quoi faire
JWT.io — Debugger	jwt.io	Inspecter et verifier ton token JWT directement dans le navigateur
jsonwebtoken (npm)	npmjs.com/package/jsonwebtoken	Doc officielle de la lib — <code>jwt.sign()</code> et <code>jwt.verify()</code>
bcrypt.js (npm)	npmjs.com/package/bcryptjs	Doc officielle — <code>bcrypt.hash()</code> et <code>bcrypt.compare()</code>
Introduction au JWT	auth0.com/learn/json-web-tokens	Article clair d'Auth0 sur le fonctionnement du JWT
Variables .env	npmjs.com/package/dotenv	Comment charger les variables d'environnement avec dotenv

Rubrique 2 — Base de donnees

Ressource	URL	Pour quoi faire
Sequelize — Migrations	sequelize.org/docs/v6/other-topics/migrations	Guide complet des migrations Sequelize CLI
Sequelize — Associations	sequelize.org/docs/v6/core-concepts/assocs	<code>hasMany</code> , <code>belongsTo</code> , <code>foreignKey</code> , <code>onDelete</code>
Sequelize — Validations	sequelize.org/docs/v6/core-concepts/validations-and-constraints	<code>isEmail</code> , <code>notEmpty</code> , <code>ENUM</code> , <code>allowNull</code>
pgAdmin 4	pgadmin.org	Interface graphique pour inspecter ta base PostgreSQL
Railway — PostgreSQL	docs.railway.app/databases/postgresql	Creer une base PostgreSQL sur Railway en 2 minutes

Rubrique 3 — API REST

Ressource	URL	Pour quoi faire
express-validator	express-validator.github.io/docs	Valider et nettoyer les donnees des requetes
Codes HTTP — MDN	developer.mozilla.org/fr/docs/Web/HTTP/Status	Reference complete des codes de statut HTTP
swagger-jsdoc	npmjs.com/package/swagger-jsdoc	Generer la spec OpenAPI depuis les commentaires JSDoc

swagger-ui-express	npmjs.com/package/swagger-ui-express	Servir l'interface Swagger sur /api-docs
Postman	postman.com	Tester toutes les routes avec collections organisees

Rubrique 4 — Interface React

Ressource	URL	Pour quoi faire
React — Context API	react.dev/learn/passing-data-deeply-with-context	Documentation officielle du Context API
React Router DOM v6	reactrouter.com/en/main	Navigation, useNavigate, useParams, route privée
Axios — Intercepteurs	axios-http.com/docs/interceptors	Ajouter le token JWT automatiquement sur chaque requete
Vite — Variables env	vitejs.dev/guide/env-and-mode	Comment utiliser VITE_API_URL dans React
Framer Motion	framer.com/motion	Animer les cartes de contacts (optionnel mais apprecie)

Rubrique 5 — Deploiemnt

Ressource	URL	Pour quoi faire
Railway — Deploiemnt	docs.railway.app/deploy/deployments	Deployer un projet Node.js sur Railway depuis GitHub
Railway — Variables env	docs.railway.app/develop/variables	Ajouter NODE_ENV, JWT_SECRET, etc. dans Railway
Vercel — React Vite	vercel.com/docs/frameworks/vite	Deployer un projet Vite+React sur Vercel en 3 clics
Vercel — Variables env	vercel.com/docs/environment-variables	Ajouter VITE_API_URL dans les variables Vercel
CORS — express	npmjs.com/package/cors	Configurer CORS correctement pour Vercel + Railway

6 · LIVRAISON ET SEUIL DE DEBLOCAGE

Ce que tu soumetts sur Moodle

Checklist de soumission — avant de cliquer Soumettre

- URL GitHub du repo (public ou prive avec acces formateur)
- URL Railway de l'API — doit repondre sur /api/health
- URL Vercel du frontend — doit permettre de creer un compte et se connecter
- README.md complete avec screenshot, URLs et instructions locales
- Le fichier .env n'est PAS dans le repo GitHub (verifie avec git log)
- Le .env.example EST dans le repo avec les cles vides

Seuil de deblocage vers le Mois 2

Score minimum pour debloquer le Devoir 2 (Mois 2)

17 / 20

85% des criteres valides

Cela signifie : au maximum 3 criteres non valides sur 20.

Si tu n'atteins pas 17/20

Tu n'as pas a tout refaire. Seulement les criteres non valides.

Le correcteur t'indique precisement lesquels (exemple : 'R3-C2 non valide — la validation email manque').

Tu corriges et tu resoumets. Pas de limite de tentatives, mais chaque resoumission doit etre sur la meme URL de production.

Le Devoir 2 ne s'ouvre sur Moodle qu'apres validation de ton score par le formateur.

Si tu atteins 17/20 ou plus — Felicitations

Le Devoir 2 (Mois 2 — Library API) se debloque automatiquement sur Moodle.

ContactBook rejoint ton portfolio GitHub public — c'est un vrai projet a montrer.

Tu as prouve que tu sais construire une application full stack de bout en bout, seul.